

ITE 14 MIDTERM EXAM WITH ANSWERS

I. Identification (12 pts)

1. A searching algorithm that checks each element from the beginning of the list until it finds a match or the whole list has been searched? (1 pt)

Answer: Linear Search

2. An ADT where elements could be added at the top of the list? (1 pt)

Answer: Stack

3. a computer programming technique involving the use of a procedure, subroutine, function, or algorithm that calls itself one or more times until a specified condition is met at which time the rest of each repetition is processed from the last one called to the first. (1 pt)

Answer: Recursion

4. It is a measurement of storage that an algorithm uses to execute properly? (1 pt)

Answer: Space Complexity

5. It is an amount of time to execute an algorithm ? (1 pt)

Answer: Time Complexity

6. Which is faster among the list: $T(n)$, $T(n^2)$, $T(n^3)$, $T(n\log n)$? (1 pt)

Answer: $T(n)$ is the fastest, followed by $T(n\log n)$, then $T(n^2)$, and finally $T(n^3)$. This ordering is based on their respective growth rates.

7. A structure containing a member that is a pointer to the same structure type? (1 ptr)

Answer: Self-referential Structure

8. The _____ of a node is the number of edges from the root to the node. (1 pt)

Answer: depth

9. A function that calls itself either directly or indirectly through another function. (1 pt)

Answer: Recursive function

II. Comprehension (12 pts)

1. What is the running time of the code given below? (1 pt)

```
x = z[2] + 4 * x;
```

Answer: constant, denoted as $O(1)$

2. What is the running time of the code given below? (2 pts)

```
for (i=0;i<n;i++) {  
    y[i]=y[i]+i;  
    z = y[i]*2;  
}
```

}

Answer: the running time of the code is **$O(n)$** .

3. What is the running time of the code given below? (3 pts)

```
for (i = 0 ; i < n ; i++)  
    for (j=0;j<i;j++)  
        if (a[j]<a[j+1]) {  
            temp=a[j];  
            a[j]=a[j+1];  
            a[j+1]=temp;  
        }  
}
```

Answer:

The calculation is as follows:

The total number of operations = $(n * (n - 1)) / 2$.

Therefore, the time complexity is $O(n^2)$.

4. In an array implementation of a queue where **front** holds the index value of the first added element and **rear** holds the index value of the last added element , if the initial value of front is - 1 for an empty queue, what is the value of rear if the queue has 1 element? (1 pt)

Answer: rear = 1

If the initial value of **front** is -1 for an empty queue and the queue has 1 element, then the value of **rear** would be 0.

This is because in a queue with only one element, both **front** and **rear** would point to the same index, which is the index of the single element in the array. Since **front** is at index -1 for an empty queue, and the queue has 1 element, the index of that element would be 0, so **rear** would also be 0.

5. Find the upper bound for $f(n) = 5n^3 + n^2 + 8$. show your solution. (3 pts)

Answer:

The calculation for the solution is as follows:

1. Choose $g(n) = 5n^3$.
2. We need to find constants $c = 6$ and $n_0 = 1$.
3. Verify that $f(n) \leq cg(n)$ for all $n \geq n_0$:
 - For $n \geq 1$:
 - $1 + \frac{1}{5} + \frac{8}{5} = 1 + \frac{9}{5} < 6$.

Therefore, the upper bound for $f(n)$ is $O(n^3)$.

6. What is the value of `*ptrX`? (2 pts)

```
int a = 10, *ptrX, *ptrY;  
ptrX = &a;  
ptrY = &ptrX;  
//Address of a is 1004, ptrX is 1008, and ptrY is 1012.
```

Answer:

```
int a = 10, *ptrX, *ptrY;  
ptrX = &a;  
ptrY = &ptrX;
```

`*ptrX = *(&a) = a`

the value of `*ptrX` is the value of `a`, which is 10.

III. Programming (36 pts)

- a. Create a function that will search an element in an array of real numbers and returns the index where the element is located and returns -1 if the element is not found (5 pts).
Sample function `x = search(A,E)` ; where `x` gets the index where the `E` is located in array `A`.

Answer:

```
#include <stdio.h>  
  
#define MAX_SIZE 100  
  
int search(int A[], int n, int E) {  
    for (int i = 0; i < n; i++) {  
        if (A[i] == E) {  
            return i; // Element found, return its index  
        }  
    }  
    return -1; // Element not found  
}  
  
int main() {  
    int arr[MAX_SIZE];  
    int size;  
  
    // Input array size  
    printf("Enter the size of the array: ");  
    scanf("%d", &size);  
  
    // Input array elements  
    printf("Enter %d elements:\n", size);
```

```

for (int i = 0; i < size; i++) {
    printf("Enter element %d: ", i + 1);
    scanf("%d", &arr[i]);
}

// Input element to search for
int element;
printf("Enter the element to search for: ");
scanf("%d", &element);

// Call search function
int index = search(arr, size, element);

// Display result
if (index != -1) {
    printf("Element %d found at index %d\n", element, index);
} else {
    printf("Element %d not found in the array\n", element);
}

return 0;
}

```

- b. Create a function that will sort the list of integers in ascending or descending order. (5 pts)
 Sample function $S = \text{sort}(A, E)$; where S is the sorted list, A is the original list, and E is the choice to sort in ascending or descending order.

Answer:

```

#include <stdio.h>

#define MAX_SIZE 100

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void sort(int A[], int n, char E) {
    // Sort the list based on the choice
    if (E == 'A') {
        // Ascending sort
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (A[j] > A[j + 1]) {
                    swap(&A[j], &A[j + 1]);
                }
            }
        }
    }
}

```

```

    }
} else if (E == 'D') {
    // Descending sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (A[j] < A[j + 1]) {
                swap(&A[j], &A[j + 1]);
            }
        }
    }
} else {
    printf("Invalid choice! Please enter 'A' for ascending or 'D' for
descending.\n");
}
}

int main() {
    int arr[MAX_SIZE];
    int size;
    char choice;
    char sortChoice;

    // Input array size
    printf("Enter the size of the array: ");
    scanf("%d", &size);

    // Input array elements
    printf("Enter %d elements:\n", size);
    for (int i = 0; i < size; i++) {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &arr[i]);
    }

    // Ask the user what they want to display
    printf("Enter 'A' to display the original list or 'S' to sort the list:
");
    scanf(" %c", &choice);

    // Display the original list or sort the list based on user's choice
    if (choice == 'A') {
        // Display the original list
        printf("Original array:\n");
        for (int i = 0; i < size; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    } else if (choice == 'S') {
        // Ask the user for sorting order
        printf("Enter 'A' for ascending sort or 'D' for descending sort: ");
        scanf(" %c", &sortChoice);

        // Sort the list
        sort(arr, size, sortChoice);
    }
}

```

```

        // Display the sorted list
        printf("Sorted array:\n");
        for (int i = 0; i < size; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    } else {
        printf("Invalid choice!\n");
    }

    return 0;
}

```

- c. Create a function that will insert into the list of characters and returns 0 if the insert is unsuccessful and 1 if the insert is successful. (5 pts)
- Sample function $x = \text{insert}(A, E)$; where x is either 0 or 1, A is the array of characters and E is the index where the element is to be inserted.

Answer:

```

#include <stdio.h>

#define MAX_SIZE 100

int insert(char A[], int size, char element, int index) {
    if (index < 0 || index > size) {
        printf("Invalid index! Insertion unsuccessful.\n");
        return 0; // Insertion unsuccessful
    }

    // Shift elements to the right to make space for the new element
    for (int i = size; i > index; i--) {
        A[i] = A[i - 1];
    }

    // Insert the new element at the specified index
    A[index] = element;

    return 1; // Insertion successful
}

int main() {
    char arr[MAX_SIZE];
    int size;
    char element;
    int index;

    // Input array size
    printf("Enter the size of the array: ");
    scanf("%d", &size);
}

```

```

// Input array elements
printf("Enter %d characters:\n", size);
for (int i = 0; i < size; i++) {
    printf("Enter character %d: ", i + 1);
    scanf(" %c", &arr[i]);
}

// Input index where the element is to be inserted
printf("Enter the index where the element is to be inserted: ");
scanf("%d", &index);

// Input element to insert
printf("Enter the character to insert: ");
scanf(" %c", &element);

// Call insert function
int result = insert(arr, size, element, index);

// Display result
if (result) {
    printf("Insertion successful.\n");
} else {
    printf("Insertion unsuccessful.\n");
}

// Display updated array
printf("Updated array:\n");
for (int i = 0; i < size + result; i++) {
    printf("%c ", arr[i]);
}
printf("\n");

return 0;
}

```

- d. Create a function that will delete an element from a stack and returns the deleted element. (5 pts) Sample function $x = \text{pop}(S)$; where x is the deleted element and S is the stack where the element is deleted. However if the stack is empty it should return -1.

Answer:

```

#include <stdio.h>

#define MAX_SIZE 100

int stack[MAX_SIZE];
int top = -1;

void push(int element) {

```

```

    if (top == MAX_SIZE - 1) {
        printf("Stack overflow! Cannot push element.\n");
        return;
    }
    stack[++top] = element;
}

int pop() {
    if (top == -1) {
        printf("Stack underflow! Cannot pop element.\n");
        return -1; // Stack is empty
    }
    return stack[top--];
}

int main() {
    int deletedElement;

    // Example usage: pushing elements onto the stack
    push(10);
    push(20);
    push(30);

    // Delete an element from the stack and return the deleted element
    deletedElement = pop();

    if (deletedElement != -1) {
        printf("Deleted element: %d\n", deletedElement);
    } else {
        printf("Stack is empty! Cannot delete element.\n");
    }

    return 0;
}

```

- e. Create a function that will add an element to a queue and return the element that was added or -1 if adding is unsuccessful (5 pts). Sample function `x = enqueue(Q,E)`; where x is the element is being added or -1 otherwise, Q is the queue where E is added.

Answer:

```

#include <stdio.h>

#define MAX_SIZE 100

int queue[MAX_SIZE];
int front = -1;
int rear = -1;

int enqueue(int Q[], int element) {
    if ((rear + 1) % MAX_SIZE == front) {

```



```

        printf("Queue overflow! Cannot add element.\n");
        return -1; // Adding unsuccessful
    }

    if (front == -1) {
        front = 0;
        rear = 0;
    } else {
        rear = (rear + 1) % MAX_SIZE;
    }

    Q[rear] = element;
    return element; // Adding successful
}

int main() {
    int elementToAdd;
    int addedElement;

    // Example usage: adding elements to the queue
    enqueue(queue, 10);
    enqueue(queue, 20);
    enqueue(queue, 30);

    // Add an element to the queue and return the added element or -1 if
    adding is unsuccessful
    printf("Enter the element to add to the queue: ");
    scanf("%d", &elementToAdd);

    addedElement = enqueue(queue, elementToAdd);

    if (addedElement != -1) {
        printf("Element added to the queue: %d\n", addedElement);
    } else {
        printf("Queue is full! Cannot add element.\n");
    }

    return 0;
}

```

- f. Create a function that will transfer all elements of a queue into a stack. (11 pts) Sample function $S = \text{transfer}(Q)$; where S is the stack of elements and Q is the queue of elements.

Answer:

```

#include <stdio.h>

#define MAX_SIZE 100

// Stack
int stack[MAX_SIZE];

```

```

int top = -1;

// Queue
int queue[MAX_SIZE];
int front = -1;
int rear = -1;

// Function prototypes
void push(int element);
int pop();
int dequeue();
int isEmptyQueue();
void enqueue(int Q[], int element); // Function prototype for enqueue

// Transfer function
void transfer(int Q[]) {
    while (!isEmptyQueue()) {
        int element = dequeue(Q);
        push(element);
    }
}

// Function to push an element onto the stack
void push(int element) {
    if (top == MAX_SIZE - 1) {
        printf("Stack overflow! Cannot push element.\n");
        return;
    }
    stack[++top] = element;
}

// Function to pop an element from the stack
int pop() {
    if (top == -1) {
        printf("Stack underflow! Cannot pop element.\n");
        return -1; // Stack is empty
    }
    return stack[top--];
}

// Function to dequeue an element from the queue
int dequeue(int Q[]) {
    if (front == -1) {
        printf("Queue underflow! Cannot dequeue element.\n");
        return -1; // Queue is empty
    }
    int element = Q[front];
    if (front == rear) {
        front = rear = -1; // Reset front and rear pointers
    } else {
        front = (front + 1) % MAX_SIZE;
    }
    return element;
}

```

```

}

// Function to check if the queue is empty
int isEmptyQueue() {
    return front == -1;
}

// Function to enqueue an element into the queue
void enqueue(int Q[], int element) {
    if (front == -1) {
        front = 0;
    }
    rear = (rear + 1) % MAX_SIZE;
    Q[rear] = element;
}

int main() {
    // Example usage: Enqueue elements into the queue
    enqueue(queue, 10);
    enqueue(queue, 20);
    enqueue(queue, 30);

    // Transfer elements from the queue to the stack
    transfer(queue);

    // Print elements from the stack
    printf("Elements in the stack after transfer:\n");
    while (top != -1) {
        printf("%d\n", pop());
    }

    return 0;
}

```